

Разработка высокопроизводительного Windows MCP-сервера на C# (.NET 10) с поддержкой Native AOT

Внедрение протокола Model Context Protocol (MCP) кардинально изменило подход к взаимодействию больших языковых моделей (LLM) с локальным окружением пользователя. MCP стандартизирует способ, с помощью которого искусственный интеллект получает доступ к инструментам, базам данных и файловым системам на конечном устройстве.¹ Однако большинство существующих реализаций MCP-серверов базируется на интерпретируемых средах выполнения, таких как Node.js или Python, что накладывает жесткие ограничения на скорость холодного старта, потребление оперативной памяти и общую задержку при обработке системных вызовов.³

Для создания по-настоящему эффективного инструмента системного управления операционной системой Windows требуется технологический стек, обеспечивающий прямой доступ к Win32 API, максимальное быстродействие и минимальный объем потребляемых ресурсов. Использование платформы .NET 10 совместно с технологией Ahead-of-Time компиляции (Native AOT) позволяет компилировать C#-приложение непосредственно в машинный код, исключая из цепочки выполнения Just-In-Time (JIT) компилятор и тяжелую виртуальную машину .NET Runtime.⁴ Данный отчет представляет собой детальное архитектурное руководство по проектированию и разработке высокопроизводительного MCP-сервера для управления Windows.

Анализ аналогичных MCP-проектов автоматизации и управления

Перед проектированием архитектуры решения на C# необходимо провести сравнительный анализ существующих систем MCP-автоматизации и выявить их конструктивные недостатки и архитектурные ограничения.

- **MCP Desktop Automation (на базе RobotJS):** Данный сервер предоставляет базовые возможности симуляции ввода и захвата экрана с использованием Node.js.³ Основной недостаток заключается в ограничениях библиотеки RobotJS, которая имеет проблемы с совместимостью на современных сборках ОС.⁶ Кроме того, из-за лимита размера ответа в 1 МБ в стандартном протоколе MCP, высокоразрешающие скриншоты приводят к сбоям передачи данных, что вынуждает принудительно уменьшать разрешение экрана до уровня 800×600 .³
- **CoDriver MCP:** Это специализированный проект для macOS, написанный на Swift.⁶ Он решает проблемы эмуляции ввода путем замены RobotJS на нативный

фреймворк CGEvent, что обеспечивает минимальные задержки при эмуляции клавиатуры и мыши.⁶ Система активно задействует accessibility tree (дерево доступности UI) операционной системы для точного поиска элементов управления по их метаданным.⁶ Главным ограничением является жесткая привязка к macOS и отсутствие кроссплатформенности.⁷

- **OpenDesk:** Распределенная система автоматизации с открытым исходным кодом, предназначенная для управления несколькими машинами по локальной сети по зашифрованному протоколу.⁸ Несмотря на гибкость интеграции с Cursor, Claude Code и другими средами разработки, сетевая топология OpenDesk добавляет задержки на сериализацию и передачу пакетов, что снижает отзывчивость интерфейса при интерактивной работе.⁸
- **Anthropic Computer Use (встроенный API-инструмент):** Решение от создателей Claude, предназначенное для автономного визуального контроля.⁹ Оно ориентировано преимущественно на выполнение в изолированных виртуальных Linux-контейнерах с использованием виртуального X11-дисплея (Xvfb).⁹ Использование данного инструмента напрямую на реальной физической хост-системе Windows сопряжено с высокой задержкой, сложностью развертывания виртуального окружения и угрозами безопасности, такими как атаки класса Prompt Injection через захват содержимого экрана.⁹
- **Официальные референсные серверы (Filesystem, Git, Playwright):** Данные серверы демонстрируют эталонные подходы к изоляции файловых операций и управлению браузерами без использования визуального ввода.⁴ Однако они не предоставляют сквозных механизмов для одновременного контроля физического оборудования и операционной системы Windows.

В следующей таблице приведено детальное сопоставление ключевых архитектурных метрик существующих решений и проектируемого MCP-сервера на базе.NET 10 Native AOT ⁴:

Архитектурная метрика	RobotJS MCP	CoDriver MCP	OpenDesk	Anthropic Computer Use	C# (.NET 10) Native AOT
Базовая технология	Node.js / C++	Swift / CoreGraphics	Rust / TypeScript	Python / X11 / Xvfb	C# /.NET 10 Native ¹³
Операционная система	Кроссплатформенный	macOS	Кроссплатформенный	Linux	Windows (Win32 API) ⁵

Холодный старт	~500 мс	~150 мс	~250 мс	>1000 мс	~100 мс ⁴
Потребление RAM	50–120 МБ	20–40 МБ	30–60 МБ	>150 МБ (в контейнере)	10–15 МБ ⁴
Передача скриншотов	Лимит 1 МБ ³	Base64 PNG	Высокий трафик	Ограничено API	Оптимизированный буфер ¹⁷
Эмуляция ввода	Эмуляция через Node	Нативная (CGEvent) ⁶	Эмуляция через Rust	Эмуляция (X11) ⁹	Нативная (SendInput) ¹⁸

Архитектура.NET 10 и компиляция Native AOT для системного управления

Платформа.NET 10 предоставляет встроенные инструменты для компиляции приложений в Native AOT, что позволяет обойти классические ограничения управляемого кода.⁵

КомпиляторAhead-of-Time преобразует промежуточный IL-код в оптимизированные инструкции целевого процессора (x64), полностью удаляя неиспользуемые метаданные и библиотеки в процессе агрессивного тримминга.⁵ Это обеспечивает мгновенный запуск сервера, что крайне важно, когда MCP-клиент (например, Claude Desktop) запускает процесс сервера при каждом обращении.²⁰

Для распространения и запуска таких высокопроизводительных серверов в.NET 10 применяется встроенный инструмент dnx, позволяющий быстро разворачивать самодостаточные пакеты инструментов напрямую из NuGet-репозитория без предварительной установки локальной среды выполнения.NET.²⁰

Основной проблемой при интеграции Native AOT с существующими реализациями протокола MCP является сериализация данных.⁵ Традиционные библиотеки JSON используют рефлексии времени выполнения (Runtime Reflection) для динамического построения схем объектов.¹⁶ В сценарии Native AOT динамическая рефлексия недоступна, а вызовы методов генерации динамического кода (таких как System.Reflection.Emit) вызывают фатальные исключения на этапе исполнения.⁵

Для обхода этого ограничения в.NET 10 используется механизм генераторов исходного кода (Source Generators).¹⁶ Разработчик создает контекст сериализации

JsonSerializerContext, который генерирует код сериализации во время компиляции.²³ В официальном C# SDK для Model Context Protocol (ModelContextProtocol версии v1.3.0 и выше) реализована поддержка Native AOT и предоставлена возможность объединения пользовательских схем с внутренними типами протокола через цепочку разрешения типов

TypeInfoResolverChain.²³



Для корректной сборки MCP-сервера в режиме Native AOT файл описания проекта .csproj должен содержать параметры компиляции, запрещающие небезопасные с точки зрения тримминга динамические вызовы ⁵:

```
XML
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0-windows</TargetFramework>
    <Nullable>enable</Nullable>
```

```

<ImplicitUsings>    </ImplicitUsings>

<PublishAot>    </PublishAot>
<IsAotCompatible>    </IsAotCompatible>

<PublishTrimmed>    </PublishTrimmed>
<TrimMode>    </TrimMode>
<PublishSelfContained>    </PublishSelfContained>

<OptimizationPreference>    </OptimizationPreference>
<IlcSingleThreaded>    </IlcSingleThreaded>

<WarningsAsErrors> 2026;12055;2067;12072;12    </WarningsAsErrors>
</PropertyGroup>

<ItemGroup>
  <PackageReference Include="ModelContextProtocol" Version="1.3.0" />
  <PackageReference Include="SkiaSharp" Version="3.0.0-preview" />
</ItemGroup>
</Project>

```

Системная интеграция мультимониторного вывода и математика обратного пересчета координат

При эмуляции управления рабочим столом на многомониторных конфигурациях возникает фундаментальная проблема пространственного ориентирования искусственного интеллекта.²⁵ Большинство LLM обучено работать с одиночным изображением

фиксированного размера (обычно 1024×768 или 1280×720).⁹ В реальных условиях физическое пространство рабочего стола Windows представляет собой единый **виртуальный экран** (Virtual Screen), объединяющий все подключенные мониторы в единую систему координат, которая может содержать отрицательные смещения.²⁶

Топология многомониторных конфигураций

Windows организует экранное пространство так, что левый верхний угол основного

(Primary) монитора всегда находится в точке $(0, 0)$.²⁷ Если дополнительный монитор расположен слева или сверху от основного, координаты его пикселей будут иметь отрицательные значения.²⁶

Для получения актуальной информации обо всех активных экранах используется Win32 API EnumDisplayMonitors.²⁹ Однако прямая передача стандартных C#-делегатов в качестве колбэков unmanaged-кода приведет к падению приложения в Native AOT из-за невозможности динамического создания маршалирующих заглушек (Marshalling Stubs) в

runtime.⁵ Для обхода этого ограничения необходимо использовать безопасные указатели на статические методы, помеченные атрибутом [UnmanagedCallersOnly].¹⁵

Алгоритм захвата и оптимизации скриншотов

Для предотвращения артефактов захвата (получения черных окон вместо содержимого браузеров или приложений с аппаратным ускорением GPU) классический захват контекста GDI не подходит.³¹ Требуется выполнять побитовое копирование структуры виртуального рабочего стола через функцию BitBlt с флагом CAPTUREBLT (

`0x40000000`), который заставляет графическую подсистему принудительно блендить слои композиции Desktop Window Manager (DWM).³²

Поскольку использование стандартной библиотеки System.Drawing.Common в средах Native AOT приводит к сбоям инициализации графического движка GDI+ (PlatformNotSupportedException на этапе выполнения)¹⁷, для оптимизации изображений применяется библиотека **SkiaSharp**.¹⁷ Алгоритм захвата выглядит следующим образом:

1. Вызывается EnumDisplayMonitors для сбора структуры физических координат экранов.²⁹
2. Модель выбирает целевой экран по его индексу.
3. Через нативный графический контекст выполняется BitBlt выбранного региона в буфер памяти.³²
4. С помощью SkiaSharp полученная растровая карта масштабируется по алгоритму бикубической интерполяции до разрешения 1024×576 (сохраняя пропорции $16 : 9$) для минимизации размера payload и ускорения распознавания моделью.³

Математический аппарат обратного пересчета координат

Когда LLM анализирует скриншот, оптимизированный до разрешения

$W_{scaled} \times H_{scaled}$, и принимает решение совершить клик в точке (X_{scaled}, Y_{scaled}) , сервер обязан пересчитать эти координаты обратно в физическую систему координат

выбранного монитора M_i , а затем транслировать их в единое пространство виртуального рабочего стола Windows.²⁵

Пусть выбранный монитор M_i имеет физические границы, полученные из структуры MONITORINFOEX²⁷:

- L_i (Left) — физическая координата левой границы на виртуальном рабочем столе.
- T_i (Top) — физическая координата верхней границы на виртуальном рабочем столе.
- R_i (Right) — координата правой границы.

- B_i (Bottom) — координата нижней границы.

Тогда физическая ширина монитора составляет $W_i = R_i - L_i$, а высота — $H_i = B_i - T_i$.²⁷

Шаг 1: Масштабирование координат на плоскости физического монитора

Сначала координаты сжатого скриншота масштабируются до физического разрешения целевого монитора:

$$X_{local} = \left\lfloor X_{scaled} \times \frac{W_i}{W_{scaled}} \right\rfloor$$

$$Y_{local} = \left\lfloor Y_{scaled} \times \frac{H_i}{H_{scaled}} \right\rfloor$$

Шаг 2: Трансляция в абсолютные координаты виртуального рабочего стола

Далее координаты смещаются с учетом физического расположения выбранного монитора в пространстве виртуального рабочего стола:

$$X_{virtual} = X_{local} + L_i$$

$$Y_{virtual} = Y_{local} + T_i$$

Шаг 3: Нормализация для эмулятора ввода SendInput

Win32-функция SendInput при отправке абсолютных координат мыши требует, чтобы значения лежали в диапазоне от 0 до 65535 по осям X и Y , где точка (0, 0) соответствует левому верхнему углу виртуального рабочего стола, а (65535, 65535) — правому нижнему.²⁸

Пусть полные размеры виртуального рабочего стола составляют:

- $W_{total} = \text{GetSystemMetrics}(SM_CXVIRTUALSECREEN)$ 25
- $H_{total} = \text{GetSystemMetrics}(SM_CYVIRTUALSECREEN)$ 25
- $L_{total} = \text{GetSystemMetrics}(SM_XVIRTUALSECREEN)$ 26
- $T_{total} = \text{GetSystemMetrics}(SM_YVIRTUALSECREEN)$ 26

Тогда нормализованные координаты X_{input} и Y_{input} для структуры MOUSEINPUT рассчитываются по следующим формулам²⁵:

$$X_{input} = \left\lfloor \frac{(X_{virtual} - L_{total}) \times 65535}{W_{total} - 1} \right\rfloor$$

$$Y_{input} = \left\lfloor \frac{(Y_{virtual} - T_{total}) \times 65535}{H_{total} - 1} \right\rfloor$$

Применение данного математического пересчета гарантирует, что клик попадет в ту же физическую точку на экране пользователя, которую выбрал искусственный интеллект на сжатом изображении.²⁵

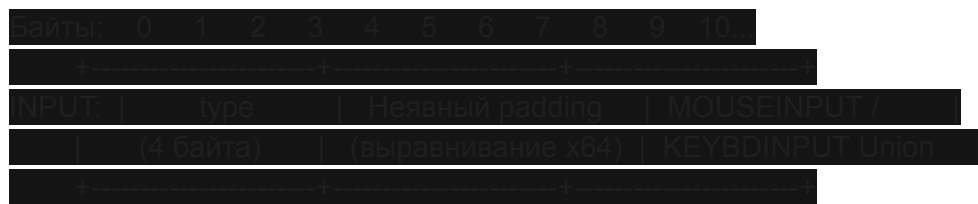
Низкоуровневая эмуляция устройств ввода через Win32 API

Для симуляции физических действий пользователя в Windows используется функция SendInput из библиотеки user32.dll.¹⁸ Она вставляет события мыши и клавиатуры непосредственно в поток системного ввода, обеспечивая обход защиты многих современных приложений, которые блокируют эмуляцию через устаревшие функции mouse_event или сообщения WM_LBUTTONDOWN.¹⁹

Проблема выравнивания структур (Alignment) в 64-битных системах

При написании P/Invoke сигнатур для Native AOT критически важно соблюдать двоичную совместимость по расположению полей в памяти (Layout).¹⁵ Структура INPUT в C++ представляет собой объединение (Union), содержащее общую часть type (4 байта) и одну из вложенных структур ввода (MOUSEINPUT, KEYBDINPUT или HARDWAREINPUT).¹⁹ На 64-битных (x64) архитектурах действует правило выравнивания по границе 8 байт (размер указателя IntPtr dwExtraInfo, входящего в состав структур ввода).⁴¹ Это означает, что компилятор Windows автоматически добавляет 4 байта неявного заполнения (padding) сразу после поля type.⁴²

Таким образом, смещение (FieldOffset) для вложенных структур mi и ki в объединении INPUT должно начинаться строго с 8-го байта, а не с 4-го.⁴² Пренебрежение этим правилом ведет к фатальным ошибкам памяти или крашу процесса при вызове SendInput на x64-системах.⁴²



Высокоточная эмуляция клавиатуры по скан-кодам

Многие современные интерактивные приложения игнорируют виртуальные коды клавиш (Virtual Key Codes, wVk) во избежание перехвата ввода.³⁸ Они считывают аппаратные скан-коды клавиш клавиатуры напрямую из драйвера контроллера (wScan).³⁸ Для обеспечения максимальной совместимости при отправке событий клавиатуры сервер должен выполнять трансляцию виртуальных клавиш в скан-коды через MapVirtualKey и передавать флаг KEYEVENTF_SCANCODE³⁸:

```
C#
public static void ExecuteKeyPress(ushort virtualKeyCode)
{
    uint scanCode = NativeInput.MapVirtualKey(virtualKeyCode, 0 // MAPVK_VK_TO_VSC

    INPUT inputs = new INPUT

    // Нажатие клавиши (Key Down)
    inputs = new INPUT

    type = NativeInput.INPUT_KEYBOARD
    ki = new KEYBDINPUT

    wVk = 0 // Игнорируем виртуальный код
    wScan = (ushort) scanCode
    dwFlags = NativeInput.KEYEVENTF_SCANCODE
    time = 0
    dwExtraInfo = IntPtr.Zero
}
```

```

// Отпускание клавиши (Key Up)
inputs = new INPUT
{
    type = NativeInput.INPUT_KEYBOARD,
    ki = new KEYBDINPUT
    {
        wVk = 0,
        wScan = (ushort)scanCode,
        dwFlags = NativeInput.KEYEVENTF_SCANCODE |
NativeInput.KEYEVENTF_KEYUP,
        time = 0,
        dwExtraInfo = IntPtr.Zero
    }
}

NativeInput.SendInput((uint)inputs.Length, inputs, Marshal.SizeOf<INPUT>());

```

Защищенные файловые операции и механизмы изоляции доступа

Поскольку МСР-сервер предоставляет ИИ-агентам непосредственный доступ к файловой системе компьютера, безопасность является критическим фактором.⁹ Неконтролируемое чтение или запись файлов может привести к утечке конфиденциальной информации или выполнению вредоносного ПО.¹¹

Песочница (Sandboxing) файлового доступа

Для защиты системы необходимо реализовать жесткое ограничение дискового пространства, выделив выделенную корневую папку ("песочницу") для работы агента.¹² Любые входящие запросы путей к файлам должны преобразовываться в абсолютные пути с последующей валидацией.¹²

Одной из самых частых атак на подобные системы является атака обхода каталога (Directory Traversal Attack), когда злоумышленник пытается выйти за пределы рабочей директории с помощью последовательностей символов .. или символических ссылок.¹² Использование API.NET 10 позволяет решить эту проблему на этапе разрешения путей с помощью класса Path без снижения производительности компиляции Native AOT.

```

C#
public class FilesystemSandbox
{
    private readonly string sandboxedDirectory;

    public FilesystemSandbox(string rootPath)
    {
        // Разрешаем абсолютный путь к выделенному каталогу
        sandboxedDirectory = Path.GetFullPath(rootPath);
    }

    public string ResolveSecurePath(string relativeOrAbsolutePath)
    {
        // Комбинируем и разрешаем относительные переходы (например,
        // /workspace/./windows/system32 -> /windows/system32)
        string resolvedPath = Path.GetFullPath(Path.Combine(sandboxedDirectory,
            relativeOrAbsolutePath));

        // Проверяем, начинается ли итоговый путь с пути нашей песочницы
        if (!resolvedPath.StartsWith(sandboxedDirectory,
            StringComparison.OrdinalIgnoreCase))
        {
            throw new UnauthorizedAccessException("Критическая ошибка безопасности: "
                + "Попытка несанкционированного доступа за пределы песочницы.");
        }

        return resolvedPath;
    }
}

```

Полный архитектурный листинг высокопроизводительного MCP-сервера

Ниже представлена готовая к компиляции архитектурная структура MCP-сервера для Windows на C# (.NET 10), адаптированная под компиляцию Native AOT.⁴ Код содержит нативные P/Invoke декларации, структуры с учетом 64-битного выравнивания и инициализацию MCP-сервера без использования динамической рефлексии.⁵

Описание низкоуровневых Win32 структур и методов (NativeMethods.cs)

Для генерации нативных P/Invoke-оберток во время компиляции используется современный атрибут [LibraryImport] вместо классического `DllImport`.¹⁵

```
C#
```

```
using System;
```

```
using System.Runtime.InteropServices;
```

```
namespace WindowsMcpServer.Native
```

```
public struct RECT
```

```
{
```

```
    public int Left;
```

```
    public int Top;
```

```
    public int Right;
```

```
    public int Bottom;
```

```
    public int Width => Right - Left;
```

```
    public int Height => Bottom - Top;
```

```
}
```

```
public struct MONITORINFOEX
```

```
{
```

```
    public int Size;
```

```
    public RECT MonitorRect;
```

```
    public RECT WorkRect;
```

```
    public uint Flags;
```

```
    public string DeviceName;
```

```
}
```

```
public struct MOUSEINPUT
```

```
{
```

```
    public int dx;
```

```
    public int dy;
```

```
    public uint mouseData;
```

```
    public uint dwFlags;
```

```
    public uint time;
```

```
public IntPtr dwExtraInfo  
|
```

```
public struct KEYBDINPUT  
|  
public ushort wVk  
public ushort wScan  
public uint dwFlags  
public uint time  
public IntPtr dwExtraInfo  
|
```

```
public struct INPUT  
|  
FieldOffset(0)  
public uint type
```

```
// Смещение 8 байт гарантирует корректную работу на x64-системах  
FieldOffset(8)  
public MOUSEINPUT mi
```

```
FieldOffset(8)  
public KEYBDINPUT ki  
|
```

```
public static partial class NativeInput  
|
```

```
public const uint INPUT_MOUSE = 0  
public const uint INPUT_KEYBOARD = 1
```

```
public const uint MOUSEEVENTF_MOVE = 0x0001  
public const uint MOUSEEVENTF_LEFTDOWN = 0x0002  
public const uint MOUSEEVENTF_LEFTUP = 0x0004  
public const uint MOUSEEVENTF_ABSOLUTE = 0x8000  
public const uint MOUSEEVENTF_VIRTUALDESK = 0x4000
```

```
public const uint KEYEVENTF_KEYUP = 0x0002  
public const uint KEYEVENTF_SCANCODE = 0x0008
```

```
public const int SM_CXVIRTUALSCREEN = 78  
public const int SM_CYVIRTUALSCREEN = 79
```

```
public const int SM_XVIRTUALSCREEN = 76
```

```
public const int SM_YVIRTUALSCREEN = 77
```

```
public static partial uint SendInput(uint cInputs, [In] INPUT pInputs, int cbSize)
```

```
public static partial uint MapVirtualKey(uint uCode, uint uMapType)
```

```
public static partial int GetSystemMetrics(int nIndex)
```

Настройка контекста сериализации (McpSerializationContext.cs)

Для работы JSON-сериализации без рефлексии в условиях тримминга Native AOT объявляется строго типизированный контекст.¹⁶

```
C#
```

```
using System.Text.Json.Serialization;
```

```
using WindowsMcpServer.Native;
```

```
namespace WindowsMcpServer.Serialization
```

```
public class MouseClickPayload
```

```
public int MonitorIndex { get; set; }
```

```
public double X { get; set; } // От 0 до 65535 (нормализованные координаты)
```

```
public double Y { get; set; }
```

```
public class FileOperationPayload
```

```
public string Path { get; set; } = string.Empty;
```

```
public string Content { get; set; }
```

```

public class ScreenCapturePayload
{
    public int MonitorIndex { get; set; }
}

```

```

internal partial class McpSerializationContext : JsonSerializerContext
{
}

```

Инициализация сервера и регистрация инструментов (Program.cs)

Реализация логики запуска, связывания сериализаторов и экспорта системных вызовов в качестве инструментов MCP.¹³

```

C#
using System;
using System.IO;
using System.Text.Json;
using System.Threading.Tasks;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using ModelContextProtocol;
using WindowsMcpServer.Native;
using WindowsMcpServer.Serialization;

namespace WindowsMcpServer

{
    public class Program
    {
        private static readonly FilesystemSandbox Sandbox = new "C:\\McpWorkspace"

        public static async Task Main(string args)
    }
}

```

```

// Создаем рабочую папку, если она не существует
if (!Directory.Exists("C:\\McpWorkspace"))
{
    Directory.CreateDirectory("C:\\McpWorkspace");
}

var builder = Host.CreateEmptyApplicationBuilder(new
HostApplicationBuilderSettings());

// Настраиваем AOT-совместимый сериализатор
builder.Services.Configure<JsonSerializerOptions>(options =>
{
    options.TypeInfoResolverChain.Insert(0,
McpJsonUtilities.DefaultOptions.TypeInfoResolver);
    options.TypeInfoResolverChain.Add(McpSerializationContext.Default);
});

// Настраиваем транспортный протокол STDIO
builder.Services.AddMcpServer()
    .WithStdioServerTransport()
    .WithToolsFromAssembly();

var app = builder.Build();
await app.RunAsync();

public static Task<string> ListMonitors()
{
    var monitors = MonitorEnumerator.GetActiveMonitors();
    string json = JsonSerializer.Serialize(monitors,
McpSerializationContext.Default.ListMONITORINFOEX);
    return Task.FromResult(json);
}

public static Task<string> ClickMouse(MouseClickPayload payload)
{
    var monitors = MonitorEnumerator.GetActiveMonitors();
    if (payload.MonitorIndex < 0 || payload.MonitorIndex >= monitors.Count)
    {
        return Task.FromResult("Ошибка: Неверный индекс монитора.");
    }
}

```



```

var targetMonitor = monitors[payload.MonitorIndex]

// 1. Вычисляем физические размеры выбранного монитора
int widthPhys = targetMonitor.MonitorRect.Right - targetMonitor.MonitorRect.Left
int heightPhys = targetMonitor.MonitorRect.Bottom - targetMonitor.MonitorRect.Top

// 2. Обратный пересчет нормализованных координат LLM во внутренние физические
координаты экрана
double xPhysLocal = (payload.X * (widthPhys - 1)) / 65535.0
double yPhysLocal = (payload.Y * (heightPhys - 1)) / 65535.0

// 3. Трансляция в абсолютные координаты виртуального рабочего стола Windows
double xVirtual = xPhysLocal + targetMonitor.MonitorRect.Left
double yVirtual = yPhysLocal + targetMonitor.MonitorRect.Top

// 4. Нормализация для SendInput с учетом размеров виртуального рабочего стола
int wTotal = NativeInput.GetSystemMetrics(NativeInput.SM_CXVIRTUALSCREEN)
int hTotal = NativeInput.GetSystemMetrics(NativeInput.SM_CYVIRTUALSCREEN)
int iTotal = NativeInput.GetSystemMetrics(NativeInput.SM_XVIRTUALSCREEN)
int jTotal = NativeInput.GetSystemMetrics(NativeInput.SM_YVIRTUALSCREEN)

int xInput = (int)((xVirtual - iTotal) / 65536.0 * wTotal)
int yInput = (int)((yVirtual - jTotal) / 65536.0 * hTotal)

// Формируем события для SendInput
INPUT inputs = new INPUT
inputs = new INPUT
{
    type = NativeInput.INPUT_MOUSE,
    mi = new MOUSEINPUT
    {
        dx = xInput,
        dy = yInput,
        dwFlags = NativeInput.MOUSEEVENTF_ABSOLUTE |
NativeInput.MOUSEEVENTF_VIRTUALDESK | NativeInput.MOUSEEVENTF_MOVE,
        NativeInput.MOUSEEVENTF_LEFTDOWN,
        mouseData = 0,
        time = 0,
        dwExtraInfo = IntPtr.Zero
    }
}

```

```

        inputs = inputs;
        inputs.mf.dwFlags = NativeInput.MOUSEEVENTF_ABSOLUTE |
NativeInput.MOUSEEVENTF_VIRTUALDESK | NativeInput.MOUSEEVENTF_LEFTUP;

        NativeInput.SendInput((uint)inputs.Length, inputs, Marshal.SizeOf<INPUT>());

        return Task.FromResult($"Клик успешно выполнен в физических координатах виртуального стола: ({xVirtual}, {yVirtual}).");
    }
}

```

```

    public static async Task<string> ReadSandboxFile(FileOperationPayload payload)
    {
        try
        {
            // Валидируем путь во избежание Directory Traversal
            string securePath = Sandbox.ResolveSecurePath(payload.Path);
            if (!File.Exists(securePath))
            {
                return "Ошибка: Файл не существует."
            }

```

```

            string text = await File.ReadAllTextAsync(securePath);
            return text;
        }
        catch (Exception ex)
        {
            return $"Ошибка доступа к файлу: {ex.Message}"
        }
    }
}

```

```

    public static async Task<string> WriteSandboxFile(FileOperationPayload payload)
    {
        try
        {
            string securePath = Sandbox.ResolveSecurePath(payload.Path);
            await File.WriteAllTextAsync(securePath, payload.Content ?? string.Empty);
            return $"Файл '{payload.Path}' успешно записан."
        }
        catch (Exception ex)
        {

```

```
return $"Ошибка при записи файла: {ex.Message}"
```

Безопасное перечисление мониторов на низком уровне (MonitorEnumerator.cs)

Реализация сбора структуры экранов без утечек памяти и в соответствии с ограничениями AOT-компиляции.²⁹

```
C#
```

```
using
```

```
using System.Collections.Generic;
```

```
using System.Runtime.InteropServices;
```

```
using WindowsMcpServer.Native;
```

```
namespace WindowsMcpServer
```

```
public unsafe class MonitorEnumerator
```

```
{
```

```
    public static partial bool EnumDisplayMonitors(
```

```
        IntPtr hdc,
```

```
        IntPtr lprcClip,
```

```
        delegate* unmanaged<IntPtr, IntPtr, RECT*, IntPtr, int> lpfnEnum,
```

```
        IntPtr dwData
```

```
    )
```

```
    public static partial bool GetMonitorInfo(IntPtr hMonitor, ref MONITORINFOEX lpmi)
```

```
    private static readonly List<MONITORINFOEX> Monitors = new
```

```
    {
```

```
        [UnmanagedCallersOnly]
```

```

    public static unsafe int MonitorEnumCallback(IntPtr hMonitor, IntPtr hdcMonitor, RECT*
    lpRectMonitor, IntPtr dwData)
    {
        MONITORINFOEX info = new
        {
            info.Size = Marshal.SizeOf<MONITORINFOEX>()
        };
        if (GetMonitorInfo(hMonitor, ref info))
        {
            Monitors.Add(info);
        }

        return 1 // Продолжить перечисление
    }

    public static List<MONITORINFOEX> GetActiveMonitors()
    {
        Monitors.Clear();

        unsafe
        {
            // Передаем указатель на статический метод, совместимый с Native AOT [30]
            EnumDisplayMonitors(IntPtr.Zero, IntPtr.Zero, &MonitorEnumCallback, IntPtr.Zero);
        }

        return Monitors;
    }
}

```

Источники

1. Get started with .NET AI and MCP - Microsoft Learn, дата последнего обращения: мая 20, 2026, <https://learn.microsoft.com/en-us/dotnet/ai/get-started-mcp>
2. Add and manage MCP servers in VS Code, дата последнего обращения: мая 20, 2026, <https://code.visualstudio.com/docs/copilot/customization/mcp-servers>
3. MCP Desktop Automation - Servers, дата последнего обращения: мая 20, 2026, <https://mcp-servers.org/servers/tanob/mcp-desktop-automation>
4. NET MCP Server 2 - LobeHub, дата последнего обращения: мая 20, 2026, <https://lobehub.com/mcp/jamesburton-dotnetmcpserver2>
5. Native AOT deployment overview - .NET | Microsoft Learn, дата последнего обращения: мая 20, 2026, <https://learn.microsoft.com/en-us/dotnet/core/deploying/native-aot/>
6. I built an MCP server that lets Claude control your entire desktop (just shipped macOS Sequoia fix!) - Reddit, дата последнего обращения: мая 20, 2026, https://www.reddit.com/r/ClaudeAI/comments/1r2qza4/i_built_an_mcp_server_that_lets_claude_control/
7. Let Claude use your computer from the CLI - Claude Code Docs, дата последнего обращения: мая 20, 2026,

- <https://code.claude.com/docs/en/computer-use>
8. Computer-use MCP that can control multiple machines (Integrate with claude, Cursor, Codex or your custom harness) : r/LocalLLaMA - Reddit, дата последнего обращения: мая 20, 2026,
https://www.reddit.com/r/LocalLLaMA/comments/1tcpgsv/computeruse_mcp_that_can_control_multiple/
 9. Computer use tool - Claude API Docs, дата последнего обращения: мая 20, 2026,
<https://platform.claude.com/docs/en/agents-and-tools/tool-use/computer-use-tool>
 10. Claude Computer Use: A Practical Guide for Developers (2026) - Deck, дата последнего обращения: мая 20, 2026,
<https://deck.co/blog/claude-computer-use-developer-guide>
 11. What Is Native Computer Use in AI Models? GPT-5.4 and Beyond | MindStudio, дата последнего обращения: мая 20, 2026,
<https://www.mindstudio.ai/blog/what-is-native-computer-use-ai-models>
 12. modelcontextprotocol/servers: Model Context Protocol Servers - GitHub, дата последнего обращения: мая 20, 2026,
<https://github.com/modelcontextprotocol/servers>
 13. Quickstart - Create a minimal MCP server and publish to NuGet ..., дата последнего обращения: мая 20, 2026,
<https://learn.microsoft.com/en-us/dotnet/ai/quickstarts/build-mcp-server>
 14. GitHub - modelcontextprotocol/csharp-sdk: The official C# SDK for Model Context Protocol servers and clients. Maintained in collaboration with Microsoft., дата последнего обращения: мая 20, 2026,
<https://github.com/modelcontextprotocol/csharp-sdk>
 15. Native interoperability best practices - .NET - Microsoft Learn, дата последнего обращения: мая 20, 2026,
<https://learn.microsoft.com/en-us/dotnet/standard/native-interop/best-practices>
 16. ASP.NET Core support for Native AOT - Microsoft Learn, дата последнего обращения: мая 20, 2026,
<https://learn.microsoft.com/en-us/aspnet/core/fundamentals/native-aot?view=aspnetcore-10.0>
 17. Breaking change: System.Drawing.Common only supported on Windows - .NET, дата последнего обращения: мая 20, 2026,
<https://learn.microsoft.com/en-us/dotnet/core/compatibility/core-libraries/6.0/system-drawing-common-windows-only>
 18. API -> SendInput [Library : user32.dll, Category : Keyboard] [C#, VB, VB.net] - BinaryWorld, дата последнего обращения: мая 20, 2026,
<https://binaryworld.net/Main/ApiDetail.aspx?Apild=4644>
 19. SendInput function (winuser.h) - Win32 apps - Microsoft Learn, дата последнего обращения: мая 20, 2026,
<https://learn.microsoft.com/en-us/windows/win32/api/winuser/nf-winuser-sendinput>
 20. MCP servers in NuGet packages - Microsoft Learn, дата последнего обращения: мая 20, 2026, <https://learn.microsoft.com/en-us/nuget/concepts/nuget-mcp>
 21. OllamaSharp.ModelContextProtocol 5.4.12 - NuGet Gallery, дата последнего

- обращения: мая 20, 2026,
<https://www.nuget.org/packages/OllamaSharp.ModelContextProtocol/5.4.12>
22. Surviving Native AOT: The Reflection Migration Guide Every .NET Architect Needs, дата последнего обращения: мая 20, 2026,
<https://blog.stackademic.com/surviving-native-aot-the-reflection-migration-guide-every-net-architect-needs-fa3760fbb41b>
 23. Experimental APIs | MCP C# SDK, дата последнего обращения: мая 20, 2026,
<https://csharp.sdk.modelcontextprotocol.io/experimental.html>
 24. ModelContextProtocol 1.3.0 - NuGet, дата последнего обращения: мая 20, 2026,
<https://www.nuget.org/packages/ModelContextProtocol/>
 25. Calculate normalized coordinates for SendInput() in a multi-monitor environment, дата последнего обращения: мая 20, 2026,
<https://stackoverflow.com/questions/62759122/calculate-normalized-coordinates-for-sendinput-in-a-multi-monitor-environment>
 26. Finding virtual screen width using multiple monitors and different DPI - Stack Overflow, дата последнего обращения: мая 20, 2026,
<https://stackoverflow.com/questions/66434222/finding-virtual-screen-width-using-multiple-monitors-and-different-dpi>
 27. Win32 API GetMonitorInfo() - Renascor, дата последнего обращения: мая 20, 2026,
<https://solever.tistory.com/25>
 28. MOUSEINPUT (winuser.h) - Win32 apps | Microsoft Learn, дата последнего обращения: мая 20, 2026,
<https://learn.microsoft.com/en-us/windows/win32/api/winuser/ns-winuser-mouseinput>
 29. EnumDisplayMonitors function (winuser.h) - Win32 apps - Microsoft Learn, дата последнего обращения: мая 20, 2026,
<https://learn.microsoft.com/en-us/windows/win32/api/winuser/nf-winuser-enumerate-monitors>
 30. UnmanagedCallersOnlyAttribute Class (System.Runtime.InteropServices) | Microsoft Learn, дата последнего обращения: мая 20, 2026,
<https://learn.microsoft.com/en-us/dotnet/api/system.runtime.interopservices.unmanagedcallersonlyattribute?view=net-10.0>
 31. c# - Using BitBlt to capture screenshot, how? - Stack Overflow, дата последнего обращения: мая 20, 2026,
<https://stackoverflow.com/questions/18140375/using-bitblt-to-capture-screenshot-how>
 32. Screenshot captured using BitBlt in C# results a black image on Windows 10, дата последнего обращения: мая 20, 2026,
<https://stackoverflow.com/questions/38868275/screenshot-captured-using-bitblt-in-c-sharp-results-a-black-image-on-windows-10>
 33. Capture a Screen Shot - C# sample code - developer Fusion, дата последнего обращения: мая 20, 2026,
<https://www.developerfusion.com/code/4630/capture-a-screen-shot/>
 34. System.PlatformNotSupportedException: System.Drawing.Common on Windows - Reddit, дата последнего обращения: мая 20, 2026,

<https://www.reddit.com/r/dotnet/comments/1kvt9e3/systemplatformnotsupportedexception/>

35. Enumerating Monitors on Windows in C# - DEVELOPERS.DE, дата последнего обращения: мая 20, 2026,
<https://developers.de/2023/03/08/enumerating-monitors-on-windows-in-c/>
36. How to automate mouse actions - c++ - Stack Overflow, дата последнего обращения: мая 20, 2026,
<https://stackoverflow.com/questions/28392072/how-to-automate-mouse-actions>
37. How can I send a right-click event to an AutomationElement using WPF's UI automation?, дата последнего обращения: мая 20, 2026,
<https://stackoverflow.com/questions/6554494/how-can-i-send-a-right-click-event-to-an-automationelement-using-wpfs-ui-automa>
38. Automating Key Presses with SendInput - Stack Overflow, дата последнего обращения: мая 20, 2026,
<https://stackoverflow.com/questions/23203944/automating-key-presses-with-sendinput>
39. StructLayoutAttribute Class (System.Runtime.InteropServices) | Microsoft Learn, дата последнего обращения: мая 20, 2026,
<https://learn.microsoft.com/en-us/dotnet/api/system.runtime.interopservices.structlayoutattribute?view=netframework-4.8.1>
40. c# - Send keys through SendInput in user32.dll - Stack Overflow, дата последнего обращения: мая 20, 2026,
<https://stackoverflow.com/questions/12761169/send-keys-through-sendinput-in-user32-dll>
41. User32.EnumDisplayMonitors() cannot be used without unsafe. · Issue #559 · dotnet/pinvoke - GitHub, дата последнего обращения: мая 20, 2026,
<https://github.com/dotnet/pinvoke/issues/559>
42. LayoutKind.Sequential not followed when substruct has LayoutKind.Explicit - Stack Overflow, дата последнего обращения: мая 20, 2026,
<https://stackoverflow.com/questions/16333511/layoutkind-sequential-not-followed-when-substruct-has-layoutkind-explicit>
43. StructLayoutAttribute.Pack Field (System.Runtime.InteropServices) | Microsoft Learn, дата последнего обращения: мая 20, 2026,
<https://learn.microsoft.com/en-us/dotnet/api/system.runtime.interopservices.structlayoutattribute.pack?view=netframework-4.8.1>